

Canton Names

Readable names for Canton party IDs

Canton Names turns long Canton party IDs into readable names like `alice.unverified.cns`, and gives any Canton app a drop-in field that sends to the right party every time.

The Problem

On Canton, you send value to a long, opaque party ID:

```
auth0_007c675a429eaf831f0991308d85 :: 12201abe669f8c4d5e6f7a8b9c0d1e2f3a4b5c6d7 ...
```

You copy, paste, and hope you got it right.

One wrong character → transfer fails or reaches the wrong party.

CNS (Canton Name Service) exists, but search, registration, and resolution are hard to use directly.

The Solution

Canton Names provides:

1. **Guided search & registration** — find available names, register in minutes
2. **Shareable name pages** — link + QR code for your name
3. `CnsRecipientInput` — drop-in component for any Canton app

The resolver warns on missing, expired, or unverified names **before** sending.

Who It's For

Primary: Canton app teams

Building recipient/counterparty selection:

- Wallets
- Tokenization & settlement
- Payments & treasury
- Marketplaces

Secondary: Party holders

Who want shareable identities:

- Validator operators
- App providers
- Institutional desks
- Canton Coin holders

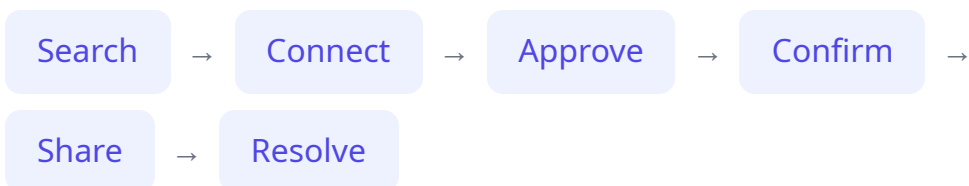
Best early adopter: A Canton app team whose users transfer value.

What's Built

Routes

/	Marketing landing
/app	Search for names
/app/register	Registration flow
/app/names	My registered names
/app/name/[name]	Details + QR
/app/demo/recipient	Integration demo

Demo Flow



RECIPIENT

Before

```
alice::1220a3f9b8c4d5e6f7a8b9c0d1e2f3a4b5c6d7e8f9a0b1c2d3e4f5a6b7c8d9e0f1c7e2
```

After

`alice.unverified.cns`

✓ Resolved

Hardest Problems Solved

Safe name-to-party resolution

- Handles nonexistent, expired, unverified, changed-between-typing-and-sending
- Validates **before** send, not after

Typed adapter layer

- Zod schemas validate all API responses
- Demo and live adapters share the same interface
- Swap adapters without changing UI code

Registration state machine

- Unit-tested transitions through all states
- Handles rejection, timeout, and retry cleanly

18 state machine tests cover all transitions

Tech Stack

Layer	Choice
Framework	Next.js 16 (App Router)
UI	React 19
Language	TypeScript (strict)
Styling	Tailwind CSS 4 + shadcn/ui
State	TanStack Query
Validation	Zod
Testing	Vitest
Package Manager	pnpm

Canton APIs

- **Scan API**
Public lookups (no auth)
- **Validator App ANS API**
Registration (JWT auth)

Also using

qrcode.react for QR codes

Validation & Proof

What we have

- **55 passing Vitest tests** (types, state machine, fixtures)
- Clean typecheck and production build
- Full demo-mode walkthrough
- Built against CNS docs and OpenAPI specs

What we don't have (yet)

- **Real users** — zero so far
- **Live registration** — untested on DevNet/TestNet
- **Wallet connection** — simulated, dApp SDK planned

This is a working demo, not a production system. Tests prove the logic; live testing proves it works on the network.

Go-to-Market

Developer-first via the resolver component.

First 10 users (target)

- Hand-held integrations with 2–3 Canton wallet/app teams
- Direct outreach via community spaces, ecosystem calls, DMs

First 100 users (target)

- 3–5 integrated apps, each bringing 20–30 registrants
- Open-source component, technical post, case study

Growth lever

The embedded resolver — every integrated app shows names and prompts unnamed recipients to claim one.

Pilot Metrics (Planned)

Metric	Purpose
Registration completion rate	Funnel health
Time to register	UX friction
Failure points by step	Drop-off analysis
Resolver lookups	Component adoption

Metric	Purpose
Recipient error reduction	Safety impact
Renewal rate	Retention signal
Integration time	Developer experience

No data yet — we need integrations to collect it.

What's Not Built Yet

Immediate (for pilot)

- Real wallet via Canton dApp SDK
- Live DevNet/TestNet registration
- Session management

Near-term

- Playwright E2E tests
- Public deployment
- Demo video
- Renewal reminders

Future

- Name transfers
- Verified names (KYC/KYB)
- Subdomain support
- Cross-app resolution SDK

How Canton Names Makes Money

Revenue model (planned)

1. Verified names for institutions

Organizations pay for identity-verified names (e.g. `bank.cns`) instead of `.unverified.cns`. Fits Canton's regulated, institutional user base.

2. Paid resolver for app teams

The drop-in `CnsRecipientInput` stays free. Paid tiers add higher lookup limits, uptime SLAs, and support for wallets, exchanges, and custodians.

3. Name registration fees

Annual renewal per name. Premium pricing for short names.

4. Canton network app rewards

Registrations and lookups generate on-chain activity that may earn app rewards (subject to current Canton reward rules).

5. Ecosystem grants

Canton Foundation / AppsFactory funding to bridge to paid tiers.

Revenue model is a plan; nothing is live yet.

The Ask

1. DevNet or TestNet access

Credentials to register names with real Canton Coin fees

2. One Canton wallet or app team willing to pilot

Embed `CnsRecipientInput` , provide feedback

That's what it takes to move Canton Names from demo mode to real transfers.